PRAYCG Formula Module Report v1.0

Full mathematical specification for the Master Comprehensive Analysis Suite v1.5.5

Status: working formula report, public-facing but technically detailed.

Scope: PRAYCG2.0 / Master Comprehensive Analysis Suite v1.5.5, including A-MRED, MRED-Peak/Resolution, NIP/CET/EET, NUPI, TTI, MRED-ITP, OCM/RSM/CVB/Squint, Topo-OSM, visualizer outputs, offline interpreter, and Opto-PING synthetic-gate references.

Central boundary: these formulas define operational indices and analysis gates. They do not prove consciousness, OSM biology, cellular hidden variables, microtubules, biophotons, memory formation, morality, or literal thermodynamic heat/energy flow.

1. Purpose of this report

This report turns the PRAYCG Master Comprehensive Analysis Suite into a formula catalog. It explains each module in enough mathematical detail that a user can understand what the suite is doing before trusting any output table.

The suite currently contains more modules than a confirmatory paper should emphasize. The recommended endpoint hierarchy is:

```
BOXED PRIMARY PATH:
  Timing/QC
  + StimulusFingerprint / CET-R
  + artifact and confound gates
  -> A-MRED / MRED-Peak-Resolution


SECONDARY SUMMARY LAYERS:
  NIP / BIT / CII / IAQ
  TTI
  NUPI
  Baseline1 vs Baseline2


EXPLORATORY / CONVERGENCE LAYERS:
```

```
KHT-topo

NAST

EET

MRED-ITP / ACG / OCU

OCM025 / RSM / CVB / SquintProxy

LSO / Subtitle Override

Opto-PING / Rung 0 synthetic gates
```

The purpose of this formula report is not to make all modules equal. It is to make every module auditable.

2. Global notation

2.1 Conditions and phases

```
C = phase-scrambled Control

T = intact Target watched naturally

O = Contextual Override: same intact narrative, analytic task stance

B1 = Baseline 1

W1, W2, W3 = washout windows after Control, Target, Override

B2 = final reflection baseline / Baseline 2
```

A time sample is indexed by `t`. A window or anchor is indexed by `j`. A cue is indexed by `i`. A frequency band is indexed by `b`.

2.2 Input streams

```
x_EEG,ch(t) = raw or cleaned EEG for channel ch

r(t)        = R-R intervals or heart-rate stream

resp(t)     = respiration stream

u(t)        = exogenous stimulus vector

m(t)        = LSL / Stasis marker stream

s(t)        = self-report / report metadata stream
```

The exogenous stimulus vector is:

```
u(t) = [audio_env(t), luminance(t), visual_change(t), optical_flow(t), cut_train(t),

        cue_train(t), cue_value(t), ALS_pulse(t), anchor_design(t)]
```

Boundary rule:

```
u(t) belongs to the stimulus / environment side.

It is not biological hidden Y(t).
```

2.3 Robust z-score

Most module scores use standardized components. The default robust z-score is:

```
rz(x_t) = (x_t - median_ref(x)) / (1.4826 * MAD_ref(x) + eps)



MAD_ref(x) = median_ref( |x_t - median_ref(x)| )
```

`eps` prevents division by zero.

A conventional z-score is:

```
z(x_t) = (x_t - mean_ref(x)) / (std_ref(x) + eps)
```

The reference distribution may be:

```
baseline only

all non-artifact windows

condition-specific windows

random-anchor null windows

pooled Control/Target/Override windows
```

The reference set must be declared by the module.

2.4 Positive clipping and logistic transform

```
clip_plus(x) = max(0, x)



sigmoid(x) = 1 / (1 + exp(-x))
```

Positive clipping is used when a module should count only positive evidence and not allow negative evidence to cancel unrelated terms.

2.5 Pre / hit / post windows

For an anchor at time `a_j`:

```
W_pre,j  = [a_j - T_pre, a_j)

W_hit,j  = [a_j - T_left, a_j + T_right]

W_post,j = (a_j + lag_start, a_j + lag_end]
```

Typical values:

```
T_pre = 10 to 30 s

T_left/T_right = 2 to 10 s around anchor

lag_start = 5 to 10 s

lag_end = 20 to 30 s
```

The exact values should be module-specific and predeclared for strict runs.

3. Data ingestion and timebase reconstruction

3.1 Marker-defined phase slicing

The runner writes marker pairs such as:

```
TARGET_1_START, TARGET_1_END

CONTROL_1_START, CONTROL_1_END

CONTEXTUAL_OVERRIDE_1_START, CONTEXTUAL_OVERRIDE_1_END

BASELINE_1_START, BASELINE_1_END

BASELINE_2_REFLECTION_START, BASELINE_2_REFLECTION_END
```

For each phase `p`:

```
phase_interval_p = [m_start,p, m_end,p]
```

A sample belongs to phase `p` if:

```
m_start,p <= t_sample < m_end,p
```

3.2 Continuous-stream reconstruction

If raw LSL timestamps are irregular, the suite can reconstruct time from status markers or sample index:

```
t_hat_n = t_anchor + (n - n_anchor) / fs_eff



fs_eff = (n_2 - n_1) / (t_2 - t_1)
```

where `(n_1, t_1)` and `(n_2, t_2)` are sample-count / LSL-time anchor pairs. The reconstructed axis is used only when it improves consistency and is logged as a timing caveat.

3.3 Branch-relative time

For rendering and anchor analysis:

```
t_rel = t_abs - t_phase_start
```

Event overlays should be repaired when mixed timebases are detected:

```
if event_time > phase_duration and phase_start_abs is known:

    event_time_rel = event_time - phase_start_abs

else:

    event_time_rel = event_time
```

4. MediaPrep formulas

MediaPrep prepares the stimulus suite. It does not certify physiology.

4.1 Cue schedule

For fixed-interval number cues:

```
t_i_start = t_start_delay + (i - 1) * T_cue_interval



t_i_end = t_i_start + T_display
```

where typical PRAYCG settings are:

```
T_cue_interval = 3.0 s

T_display = 0.85 s

cue_position = upper_right
```

The running sum is:

```
S_i = S_{i-1} + v_i

S_0 = 0
```

where `v_i` is the cue value.

4.2 Target/Override identity rule

The Target and Override media are generated from the same cue-embedded stimulus:

```
hash(Target_video) = hash(Override_video)
```

The only intended difference is participant instruction:

```
Target instruction: watch naturally.

Override instruction: perform analytic cue task.
```

4.3 Phase-scrambled Control

A simplified visual phase-scramble operation can be written in Fourier form:

```
F_target(k) = A(k) * exp(i * phi(k))

F_control(k) = A(k) * exp(i * phi_random(k))

control_frame = inverse_Fourier(F_control)
```

The intent is:

```
preserve low-level audiovisual structure

remove recognizable narrative meaning
```

The Control is generated from the cue-embedded Target so the cue rhythm and cue energy remain represented.

4.4 ALS / photodiode stimulus pulse

The ALS timing square is an exogenous display-timing regressor:

```
ALS_pulse(t) = 1 if t in [video_start, video_start + pulse_duration]

ALS_pulse(t) = 0 otherwise
```

Pass criteria include:

```
visible analog rise

return to baseline

no clipping for full pulse duration

pulse present for Control, Target, and Override
```

Boundary:

```
ALS_pulse(t) belongs to u(t), not Y_topo(t).
```

5. StimulusFingerprint v1.8

StimulusFingerprint estimates the physical stimulus structure so physiological modules can be residualized against exogenous drive.

5.1 Visual luminance

For frame `f` with pixels `P_f(x,y)` converted to grayscale intensity:

```
L_f = mean_{x,y}(gray(P_f(x,y)))
```

Resampled to time:

```
L(t) = interpolate({frame_time_f, L_f})
```

5.2 Visual change / motion proxy

A simple frame-difference proxy:

```
Vchange_f = mean_{x,y}( |gray(P_f) - gray(P_{f-1})| )
```

An optical-flow proxy may be substituted:

```
Flow_f = mean_{x,y}( sqrt(u_flow(x,y)^2 + v_flow(x,y)^2) )
```

5.3 Histogram delta

```
H_f = normalized_histogram(gray(P_f))

HistDelta_f = ||H_f - H_{f-1}||_1
```

5.4 Cut train

```
Cut_f = 1[HistDelta_f > theta_cut]
```

where `theta_cut` is calibrated from the distribution of frame changes.

5.5 Flash-event proxy

```
Flash_f = 1[ |L_f - L_{f-1}| > theta_flash ]
```

5.6 Audio RMS envelope

For audio waveform `a[n]`, sample rate `fs`, and window length `N`:

```
RMS_k = sqrt( (1/N) * sum_{n in window k} a[n]^2 )



dBFS_k = 20 * log10(RMS_k + eps)
```

Envelope derivative:

```
AudioDeriv_k = RMS_k - RMS_{k-1}
```

5.7 Cue regressors

```
CueOn(t) = 1 if any cue visible at t, else 0

CueImpulse(t) = sum_i delta(t - t_i_start)

CueValue(t) = v_i during cue i, else 0
```

```
CuePhaseSin(t) = sin(2*pi*t / T_cue_interval)

CuePhaseCos(t) = cos(2*pi*t / T_cue_interval)
```

The cue frequency is:

```
f_cue = 1 / T_cue_interval
```

For `T_cue_interval = 3.0 s`:

```
f_cue = 0.333 Hz
```

5.8 Dominant frequency map

For each stimulus regressor `u_m(t)`, compute a short-time Fourier transform:

```
U_m(f,tau) = STFT[u_m(t)]
```

Dominant local frequency:

```
f_m_star(tau) = argmax_f |U_m(f,tau)|^2
```

5.9 Anchor stimulus vector

For anchor window `W_j`:

```
StimVec_j = [mean(L), std(L), mean(Vchange), cut_rate, flash_rate,

             mean(AudioRMS), std(AudioRMS), cue_density, f_star_audio, f_star_visual]_Wj
```

This vector feeds CET/EET and stimulus-side QC.

6. Core EEG spectral features

6.1 Filtering

For a band `[f1, f2]`, a band-limited signal is:

```
x_band,ch(t) = BandPass(x_ch(t), f1, f2)
```

6.2 Welch power spectral density

For window `W_t`:

```
PSD_ch(f,t) = WelchPSD(x_ch over W_t)

BandPower_ch,b(t) = integral_{f in band b} PSD_ch(f,t) df

LogBandPower_ch,b(t) = log(BandPower_ch,b(t) + eps)
```

6.3 ROI aggregation

For ROI `R` with channels `ch in R`:

```
BandPower_R,b(t) = mean_{ch in R}(LogBandPower_ch,b(t))
```

or robust median:

```
BandPower_R,b(t) = median_{ch in R}(LogBandPower_ch,b(t))
```

6.4 Phase and phase-locking value

Using the analytic signal:

```
z_ch,b(t) = Hilbert(x_band,ch(t))

phase_ch,b(t) = angle(z_ch,b(t))
```

Pairwise phase-locking:

```
PLV_{a,b}(W) = | (1/N) * sum_{t in W} exp(i*(phase_a(t) - phase_b(t))) |
```

ROI PLV:

```
PLV_ROI(W) = mean_{channel pairs in ROI}(PLV_pair(W))
```

7. ArtifactScore

ArtifactScore is a veto/covariate layer, not a biological finding.

7.1 Peak-to-peak amplitude

```
p2p_ch(W) = max_{t in W}(x_ch(t)) - min_{t in W}(x_ch(t))
```

7.2 High-frequency sentinel

A generic muscle/line-noise sentinel:

```
HF_ch(W) = log( integral_{45 Hz}^{55 Hz} PSD_ch(f,W) df + eps )
```

Optional wider EMG sentinel:

```
EMG_ch(W) = log( integral_{55 Hz}^{95 Hz} PSD_ch(f,W) df + eps )
```

7.3 Ocular/frontal sentinel

For frontal channel group `F`:

```
FpSentinel(W) = mean_{ch in F}( z(p2p_ch(W)) + z(HF_ch(W)) ) / 2
```

7.4 Composite artifact score

Default operational form:

```
ArtifactScore(W) = mean(
  rz(p2p_all(W)),
  rz(HF_45_55(W)),
  rz(EMG_55_95(W)),
  rz(FpSentinel(W)),
  rz(line_noise_ratio(W)),
  rz(flatline_or_step_flag(W))
)
```

A stricter gate uses:

```
ArtifactPass(W) = 1[ArtifactScore(W) < theta_artifact]
```

For peak modules:

```
QC_j = ArtifactPass(W_hit,j) * TimingPass_j * ConfoundPass_j
```

8. API_A v1: autonomic availability index

API_A is a proxy for regulated availability, not valence, clinical calm, or proof of parasympathetic healing.

8.1 HR and HRV features

Given R-R intervals `RR_k` in milliseconds:

```
HR_k = 60000 / RR_k

RMSSD = sqrt( mean( (RR_{k+1} - RR_k)^2 ) )

SDNN = std(RR_k)

pNN50 = count(|RR_{k+1} - RR_k| > 50 ms) / count(pairs)
```

HR slope over a window:

```
HR_slope(W) = slope of linear regression HR(t) ~ t over W
```

8.2 API_A formula

Default operational form:

```
API_A(t) = 0.30 * z(-HR_mean(t))
         + 0.25 * z(RMSSD(t))
         + 0.20 * z(SDNN(t))
         + 0.15 * z(pNN50(t))
         + 0.10 * z(-HR_slope(t))
         - 0.10 * z(RespInstability(t))
```

If some HRV features are unavailable, weights should be renormalized over available features.

Interpretation:

```
higher API_A = lower HR / higher HRV / more regulated physiology, after respiration caution
```

9. GammaScalpel

GammaScalpel treats lower-gamma as an artifact-sensitive work-signal family, not a meaning biomarker.

9.1 Microband decomposition

Define 5 Hz bands:

```
B_1 = 30-35 Hz
B_2 = 35-40 Hz
B_3 = 40-45 Hz
B_4 = 45-50 Hz  (more artifact-sensitive)
B_5 = 50-55 Hz  (line / sentinel caution)
```

For each ROI `R` and band `b`:

```
Gamma_R,b(t) = rz(LogBandPower_R,b(t))
```

9.2 MeaningGamma proxy

MeaningGamma is a weighted candidate signal. It should not be interpreted without artifact and stimulus controls.

```
MeaningGamma(t) = sum_b w_b * [
      a_PT  * Gamma_posterior_temporal,b(t)
```

```
    + a_P   * Gamma_parietal,b(t)
    + a_O   * Gamma_occipital,b(t)
    - a_F   * Gamma_frontal_task,b(t)
]
 - lambda_art * ArtifactScore(t)
 - lambda_vis * VisualDrive(t)
 - lambda_cue * CueOn(t)
```

Typical interpretation:

```
posterior/temporal + parietal lower-gamma -> candidate semantic/scene recognition work
frontal/task gamma -> analytic extraction or task stance
visual/cue/artifact terms -> non-meaning penalties
```

9.3 TaskGamma proxy

```
TaskGamma(t) = sum_b w_b * [
      a_F  * Gamma_frontal,b(t)
    + a_C  * Gamma_central,b(t)
    + a_PLV * PLV_fronto_central,b(t)
]
 - lambda_art * ArtifactScore(t)
```

TaskGamma is expected to rise during Contextual Override when the subject is doing the running-sum task.

9.4 GammaScalpel band decision

A band is considered interpretable only if:

```
BandPass_b = 1[Artifact_b < threshold]
           * 1[LineNoise_b < threshold]
           * 1[VisualDrive_b not sufficient explanation]
```

Then:

```
MeaningGamma_valid(t) = MeaningGamma(t) * BandPass(t)
```

10. Temporal Semantic Proxy (TSP)

TSP estimates a candidate time of semantic recognition. It is not a ground-truth reading of meaning.

Operational formula specification - not mechanism proof

10.1 Components

```
Gm(t) = MeaningGamma(t)

PT(t) = posterior-temporal semantic-work proxy

N(t)  = NAST / absorption-transition support, if available

K(t)  = local KHT-topo support, if available

A(t)  = ArtifactScore(t)

V(t)  = VisualDrive(t)

X(t)  = TaskGamma(t)
```

10.2 TSP formula

```
TSP(t) = w_G  * rz(Gm(t))

       + w_PT * rz(PT(t))

       + w_N  * rz(N(t))

       + w_K  * rz(K(t))

       - w_X  * rz(X(t))

       - w_A  * rz(A(t))

       - w_V  * rz(V(t))
```

In many runs, TSP is treated as:

```
TSP_z(t) = rz(TSP(t))
```

10.3 Event peak

For anchor window `W_j`:

```
t_peak,j = argmax_{t in W_j}(TSP_z(t))

TSP_peak,j = max_{t in W_j}(TSP_z(t))
```

A strict event should not rely only on TSP. TSP identifies candidate timing; MRED/A-MRED tests whether recognition and integration survive the gates.

11. Theta handoff / integration proxy

11.1 Theta feature

```
Theta_R(t) = rz(log integral_{4 Hz}^{8 Hz} PSD_R(f,t) df)
```

Use frontal-midline / frontocentral / parietal integration ROI as configured.

11.2 Post-event theta delta

For event time `t_j`:

```
Theta_pre,j = mean(Theta_R(t) over [t_j - 10, t_j])

Theta_post_0_10,j = mean(Theta_R(t) over [t_j, t_j + 10])

Theta_post_10_30,j = mean(Theta_R(t) over [t_j + 10, t_j + 30])



H_theta_0_10,j = Theta_post_0_10,j - Theta_pre,j

H_theta_10_30,j = Theta_post_10_30,j - Theta_pre,j
```

11.3 Handoff criterion

```
ThetaHandoff_j = 1[H_theta_10_30,j > theta_handoff_threshold]
```

The delayed 10-30 s window is often more relevant than the immediate 0-10 s window when testing reflective integration rather than sensory surprise.

12. CandidateLocal_KHT-topo

CandidateLocal_KHT-topo is an event-level coupling model.

12.1 Human-scale state variables

```
E_t = observed work-signal proxy, e.g. MeaningGamma or TSP

Y_t = inferred human-translation state proxy, e.g. theta/topology/NIP state

u_t = exogenous stimulus regressors

A_t = artifact/confound regressors
```

12.2 Local reciprocal model

For local event window `W_j`:

```
E_{t+Delta} = a_E * E_t + eta_HT * Y_t + b_E^T u_t + c_E * Artifact_t + eps_E,t



Y_{t+Delta} = a_Y * Y_t + zeta_HT * E_t + b_Y^T u_t + c_Y * Artifact_t + eps_Y,t
```

where:

```
eta_HT  = Y -> E coefficient

zeta_HT = E -> Y coefficient
```

12.3 Local coupling coefficient

```
K_HT_topo,j = sqrt( abs(eta_HT,j * zeta_HT,j) )

LoopSign_j = sign(eta_HT,j * zeta_HT,j)
```

12.4 Event-lock rule

```
KLock_j = 1[K_HT_topo,j >= threshold_K or percentile_K >= threshold_percentile]



EventLock_j = KLock_j * ThetaHandoff_j * ArtifactPass_j * TimingPass_j
```

K alone is not enough. Theta handoff alone is not enough.

12.5 Model-selection guard

For candidate models:

```
M_full = reciprocal E <-> Y

M_replay = replay-only / no reciprocal hidden loop

M_eta = eta-only

M_zeta = zeta-only

M_null = no-coupling

M_generic = generic hidden oscillator
```

Use held-out prediction:

```
CVWin_j = 1[CVError_full < min(CVError_alternatives)]
```

A strong candidate requires:

```
EventLock_j * CVWin_j = 1
```

13. MRED: Meaning Recognition / Encoding Dissociation

MRED separates meaning recognition from integration/carryover.

13.1 Meaning recognition score

```
MR_j = w_TSP * TSP_peak,j

     + w_G   * MeaningGamma_peak,j

     + w_K   * KHT_topo_j

     + w_N   * NAST_j

     - w_A   * ArtifactScore_j

     - w_V   * VisualDrive_j

     - w_C   * ConfoundBurden_j
```

13.2 Encoding / integration score

```
ENC_j = w_H  * H_theta_10_30,j

      + w_T  * TopoShift_j

      + w_E  * EET_echo_j

      + w_API * API_A_post,j

      + w_B2 * Baseline2Echo_j

      - w_A  * ArtifactScore_post,j

      - w_X  * TaskGamma_post,j
```

13.3 MRED quadrant

Given thresholds `theta_MR` and `theta_ENC`:

```
if MR_j >= theta_MR and ENC_j >= theta_ENC:

    quadrant = MR_HIGH_ENC_HIGH

elif MR_j >= theta_MR and ENC_j < theta_ENC:

    quadrant = MR_HIGH_ENC_LOW

elif MR_j < theta_MR and ENC_j >= theta_ENC:

    quadrant = MR_LOW_ENC_HIGH

else:

    quadrant = MR_LOW_ENC_LOW
```

Interpretation:

```
MR_HIGH_ENC_HIGH = recognition plus integration candidate

MR_HIGH_ENC_LOW  = recognition without clear integration; can occur with familiarity

MR_LOW_ENC_HIGH  = possible nonsemantic update, task/cue/respiration/artifact caution
```

```
MR_LOW_ENC_LOW   = no detected event under current criteria
```

14. A-MRED: anchor-locked primary endpoint

A-MRED is the compressed primary endpoint.

14.1 Strict gate

For a predeclared anchor `j`:

```
A_MRED_j =

    1[AnchorLocked_j = 1]

  * 1[MR_T,j  > theta_MR]

  * 1[ENC_T,j > theta_ENC]

  * 1[MR_T,j  > MR_C,j + delta_MR]

  * 1[MR_T,j  > MR_O,j + delta_MR]

  * 1[ENC_T,j > ENC_C,j + delta_ENC]

  * 1[ENC_T,j > ENC_O,j + delta_ENC]

  * 1[QC_j = PASS]
```

where:

```
QC_j = ArtifactPass_j * TimingPass_j * ConfoundPass_j * CETRPass_j
```

14.2 Continuous backup score

```
MREDScore_condition,j = clip_plus(MR_condition,j) * clip_plus(ENC_condition,j) * QC_condition,j



DeltaMRED_j = MREDScore_T,j - max(MREDScore_C,j, MREDScore_O,j)
```

14.3 Claim-grade modifiers

```
STRICT_CONFIRMATORY:

  anchor locked before acquisition, frame verified, runner registered, ALS/timing pass.



RUNNER_REGISTERED_ESTIMATED:

  anchor loaded before acquisition but estimated/not frame verified.
```

```
CONCEPTUALLY_PREDECLARED:
  anchor existed in planning but not machine registered.

EXPLORATORY:
  event found after physiology review.
```

15. MRED-Peak vs MRED-Resolution

This module separates acute meaning shock from delayed reflective closure.

15.1 MRED-Peak

MRED-Peak asks whether an anchor produces acute Target-specific recognition and delayed integration.

```
PeakEvidence_j = mean(
    z(MR_T,j),
    z(ENC_T,j),
    z(DeltaMRED_j),
    z(CII_margin_j),
    z(KHT_topo_j)
)
```

Strict peak pass:

```
MRED_Peak_Pass_j = A_MRED_j or BIT_j
```

Soft peak candidate:

```
MRED_Peak_Candidate_j = 1[PeakEvidence_j > theta_peak] * 1[QC_j != FAIL]
```

15.2 MRED-Resolution

MRED-Resolution asks whether the event produces delayed reflective/regulatory recovery rather than a sharp event spike.

```
ResolutionEvidence_j = mean(
    z(Baseline2Echo_j),
    z(EET_j),
```

```
    z(RDI_j),

    z(TargetEchoedDuringBaseline2_report),

    z(EmotionalAfterglow_T - EmotionalAfterglow_0),

    z(API_A_B2 - API_A_B1)

)
```

Resolution candidate:

```
MRED_Resolution_Candidate_j = 1[ResolutionEvidence_j > theta_resolution]

                            * 1[TargetSpecificity_j = 1]

                            * 1[QC_j != FAIL]
```

15.3 Run-level archetype

```
if max(PeakEvidence) high and ResolutionEvidence moderate:

    archetype = MRED_PEAK

elif ResolutionEvidence high and PeakEvidence moderate:

    archetype = MRED_RESOLUTION

elif MR high and ENC low:

    archetype = RECOGNITION_DOMINANT

else:

    archetype = UNRESOLVED
```

16. NIP: Narrative Immersion Proxy

NIP operationalizes attention plus resonance without claiming dopamine, oxytocin, or proprietary immersion biology.

16.1 Semantic attention component

```
A_sem(t) = clip_plus(

      w_G * rz(MeaningGamma(t))

    + w_T * rz(TSP(t))

    + w_N * rz(NAST(t))

    - w_V * rz(VisualDrive(t))

    - w_A * rz(ArtifactScore(t))
```

```
    - w_X * rz(TaskGamma(t))

)
```

16.2 Integration / resonance component

```
R_int(t) = clip_plus(

      w_H   * rz(H_theta(t))

    + w_K   * rz(KHT_topo(t))

    + w_TOPO* rz(TopoShift(t))

    + w_API * rz(API_A(t))

    - w_A   * rz(ArtifactScore(t))

)
```

16.3 NIP density

With lag `ell`:

```
NIP_density(t) = A_sem(t)^alpha * R_int(t + ell)^beta * Penalty(t)
```

where:

```
Penalty(t) = ArtifactPass(t) * ConfoundPenalty(t) * TimingPass(t)
```

Default:

```
alpha = 1

beta = 1
```

17. BIT: Bivariate Immersion Threshold

BIT is an AND-gate, not an average.

```
BIT_j = 1[A_sem_j > theta_A]

      * 1[R_int_j > theta_R]

      * 1[Artifact_j < theta_artifact]

      * 1[Specificity_j = 1]
```

where:

```
Specificity_j = 1[Target_j > Control_j + delta]

              * 1[Target_j > Override_j + delta]
```

BIT fails when recognition is high but integration is absent:

```
MR_HIGH_ENC_LOW -> BIT = 0
```

This prevents a high attention spike from being mislabeled as full immersion.

18. CII: Continuous Immersion Index

CII measures average immersion density across an anchor window.

```
CII_condition(W_j) = (1 / |W_j|) * integral_{t in W_j} NIP_density_condition(t) dt
```

Discrete implementation:

```
CII_condition(W_j) = mean_{t in W_j}(NIP_density_condition(t))
```

Target-specific margin:

```
CII_margin_j = CII_T,j - max(CII_C,j, CII_O,j)
```

Duration normalization matters:

```
without division by |W_j|, long scenes automatically score higher.
```

19. IAQ / ITQ: Immersion Attenuation Quotient

IAQ estimates how much Contextual Override attenuated Target immersion.

```
IAQ_j = 1 - (CII_O,j + eps) / (CII_T,j + eps)
```

Equivalent difference form:

```
ITQ_j = CII_T,j - CII_O,j
```

Interpretation:

```
IAQ near 1  = Override strongly attenuated the immersion proxy

IAQ near 0  = Override preserved a similar immersion proxy

IAQ below 0 = Override exceeded Target on the immersion proxy
```

Use IAQ in technical documents; reserve “theft” language for clearly labeled philosophical/cultural discussion.

20. TTI: Reception-Extraction Tradeoff

TTI estimates whether the same narrative was received or converted into a task object.

20.1 Budget principle

```
c_R R(t) + c_X X(t) + c_M M(t) + c_Gamma Gamma(t) + c_RX R(t)X(t) <= B(t)
```

where:

```
R(t) = receptive absorption

X(t) = extractive / instrumental task stance

M(t) = semantic payload

Gamma(t) = cognitive/metabolic exhaust proxy

B(t) = finite processing budget

c_RX R(t)X(t) = collision cost
```

20.2 Dynamic model

```
dR/dt = alpha_M*M(t)*(1-R) + alpha_A*A(t)

        - beta_X*X(t)*R(t) - beta_Gamma*Gamma(t)*R(t) - lambda_R*R(t)



dX/dt = alpha_T*T(t)*(1-X) + alpha_C*Cue(t)

        - beta_R*R(t)*X(t) - lambda_X*X(t)
```

20.3 TTI composite

```
TTI = w1 * z(MR_T - MR_O)

    + w2 * z(ENC_T - ENC_O)

    + w3 * z(API_T - API_O)

    + w4 * z(X_O - X_T)

    - w5 * z(|Artifact_T - Artifact_O|)

    - w6 * z(|VisualDrive_T - VisualDrive_O|)
```

Default weights:

```
w1 MR       = 0.28

w2 ENC      = 0.24

w3 API_A    = 0.14

w4 X-load   = 0.22

w5 Artifact = 0.08 penalty
```

```
w6 Visual   = 0.04 penalty
```

Positive TTI means Target preserved more reception/integration while Override carried more extraction/task load.

21. NUPI: Narrative Update Polarity Index

NUPI asks what kind of update meaning demanded.

21.1 Accommodative Load Index (ALI)

```
ALI = mean(

    z(semantic_intensity),

    z(Target_specificity),

    z(complexity_perturbation),

    z(TTI_positive_load),

    z(primary_endpoint_pass_strength)

)
```

Interpretation:

```
High ALI = model-expansion, awe, shock, destabilizing accommodation, or high update demand.
```

21.2 Regulatory Dividend Index (RDI)

```
RDI = mean(

    z(Baseline2_regulation),

    z(Baseline2_semantic_echo),

    z(EET_afterstate_echo),

    z(TargetEchoedDuringBaseline2_selfreport),

    z(API_A_B2 - API_A_B1)

)
```

Interpretation:

```
High RDI = reflective recovery, closure, parasympathetic regulation, after-state coherence.
```

21.3 NUPI score

```
NUPI = RDI - ALI
```

Classification:

```
NUPI > theta_positive and RDI high:

  RESOLUTIVE_RECOVERY



ALI high and RDI moderate/high:

  HIGH_LOAD_WITH_RECOVERY



ALI high and RDI low:

  ACCOMMODATIVE_LOAD



MR high but ENC/RDI unclear:

  RECOGNITION_DOMINANT_OR_UNRESOLVED
```

Boundary:

```
NUPI does not measure heat, ATP, clinical healing, morality, or literal thermodynamics.
```

22. NAST: Narrative Absorption State Transition

NAST estimates transition from idle/analytic/resting state into narrative absorption.

22.1 Alpha/idling proxy

```
Alpha_R(t) = rz(log integral_{8 Hz}^{12 Hz} PSD_R(f,t) df)
```

Absorption is often expected to show alpha desynchronization relative to baseline, but this is not universal.

22.2 NAST proxy

```
NAST(t) = w_TSP * rz(TSP(t))

        + w_G   * rz(MeaningGamma(t))

        + w_H   * rz(ThetaIntegration(t))

        - w_A   * rz(Alpha(t))

        - w_X   * rz(TaskGamma(t))

        - w_art * rz(ArtifactScore(t))
```

22.3 Phase transition contrast

For transition from phase `p` to phase `q`:

```
DeltaNAST_{p->q} = mean(NAST over early q) - mean(NAST over late p)
```

Example:

```
WASHOUT_1 -> TARGET

WASHOUT_2 -> OVERRIDE
```

Interpretation:

```
positive Target transition = narrative absorption candidate

positive Override transition with TaskGamma = analytic reorientation candidate
```

23. OCM025: Override Cue Microstate analysis

OCM025 analyzes cue-locked micro-windows at 0.25 s resolution.

23.1 Cue-relative bins

For cue onset `t_i`:

```
pre_i       = [t_i - 0.50, t_i)

digit_i     = [t_i, t_i + 0.85]

update_i    = [t_i + 0.25, t_i + 1.25]

maint_i     = [t_i + 1.25, min(t_i + 3.0, t_{i+1}))
```

23.2 Digit recognition gamma

```
DR_i = mean(MeaningGamma or visual/digit gamma over digit_i)

     - mean(same feature over pre_i)
```

23.3 Working-memory update theta

```
WMU_i = mean(frontal_theta over update_i) - mean(frontal_theta over pre_i)
```

23.4 Maintenance theta

```
MAINT_i = mean(frontal_theta over maint_i) - mean(frontal_theta over pre_i)
```

23.5 OCM score

```
OCM_i = 0.35*z(DR_i)
      + 0.35*z(WMU_i)
      + 0.20*z(MAINT_i)
      - 0.10*z(Artifact_i)
```

OCM is Override-only unless explicitly used as a cue confound check.

24. RSM: Running Sum Microstate Model

RSM estimates arithmetic/cue burden during Override.

24.1 Running sum

```
S_i = S_{i-1} + v_i
```

24.2 Carry features

```
prior_units_i = S_{i-1} mod 10

carry_required_i = 1[prior_units_i + v_i >= 10]

high_carry_load_i = max(0, prior_units_i + v_i - 9)

hard9_i = 1[v_i = 9 and prior_units_i in {7,8,9}]
```

24.3 Arithmetic Compute Load

```
ACL_i = beta1*z(log(1 + S_{i-1}))
      + beta2*carry_required_i
      + beta3*z(high_carry_load_i)
      + beta4*hard9_i
      + beta5*z(fatigue_or_elapsed_time_i)
```

24.4 Compute-stall score

```
ComputeStall_i = 0.35*z(WMU_i)
               + 0.30*z(MAINT_i)
               + 0.25*z(T_close_i)
               + 0.10*OpenLoop_i
```

```
                - 0.20*z_positive(Artifact_i)
```

where:

```
T_close_i = time until theta/update loop returns below closure threshold

OpenLoop_i = 1[T_close_i > t_{i+1} - safety_buffer]
```

24.5 Latent guess-risk

```
GuessRisk_i = sigmoid(

    gamma0

  + gamma1*ArithmeticStall_i

  + gamma2*DeadlinePressure_i

  + gamma3*ACL_i

  + gamma4*CueVisibilityBurden_i

  + gamma5*PriorUncertainty_i

  - gamma6*FinalSumConfidence

)
```

This is a latent proxy, not proof that the subject guessed.

25. CVB: Cue Visibility / Legibility Burden

CVB estimates cue-reading strain.

```
CVB_i = 0.45*z(HoldBurden_i)

      - 0.25*z(DigitRecognition_i)

      + 0.15*z(VisualSearch_i)

      + 0.15*z(1 - CueContrast_i)

      + 0.15*z(CueBlurOrSmallness_report)

      - 0.10*z(Artifact_i)
```

For small/blurred cues, contrast may be high while legibility remains poor. CVB should therefore not rely only on luminance contrast.

26. SquintProxy

SquintProxy estimates frontal visual-strain artifacts. It cannot confirm squinting.

```
SquintProxy_i = 0.30*z(Fp1_HF_i)
              + 0.25*z(Fp1_p2p_i)
              + 0.20*z(frontal_EMG_proxy_i)
              + 0.15*z(EyeStrainOrSquint_report_i)
              + 0.10*z(CueBlurOrSmallness_report_i)
              - 0.20*z(BlinkTemplate_i)
```

Interpretation:

```
high SquintProxy = possible visual strain / ocular or forehead artifact

not confirmed squint
```

EOG or eye tracking is required for confirmation.

27. Confound registry

Confound reports are covariates/veto aids.

27.1 Branch confound score

```
ConfoundBurden_branch = mean(
    AudioVideoSyncProblem,
    AudioComprehensionDifficulty,
    ExternalNoiseIntrusion,
    SpeakerVolumeDifficulty,
    CueBlurOrSmallness,
    EyeStrainOrSquint
) / 9
```

27.2 Gate

```
ConfoundPass_branch = 1[ConfoundBurden_branch < theta_confound]
```

or claim-grade modifier:

```
if ConfoundBurden_branch >= theta_caution:
    claim_level = claim_level + "_CONFOUND_CAUTION"
```

Confounds should lower claim strength, not create positive evidence.

28. CET: Cinematic Entrainment Tracking

CET quantifies how much EEG tracks exogenous stimulus rhythm.

28.1 Stimulus transform

```
U_m(f,tau) = STFT[u_m(t)]
```

where `u_m` may be audio envelope, luminance, visual change, cut train, cue train, etc.

28.2 EEG transform

```
X_ch(f,tau) = STFT[x_ch(t)]
```

28.3 Coherence

```
Coh_{u,x}(f,tau) = |S_{u,x}(f,tau)|^2 / (S_{u,u}(f,tau) * S_{x,x}(f,tau) + eps)
```

28.4 Phase lag

```
phi_{u,x}(f,tau) = angle(S_{u,x}(f,tau))

lag_{u,x}(f,tau) = phi_{u,x}(f,tau) / (2*pi*f)
```

Caution:

```
Stable lag can mean good tracking, not stalled biological momentum.
```

29. CET-R: residualization against exogenous drive

CET-R asks whether MRED/NIP/TTI survives after modeling stimulus rhythm.

29.1 Regression model

For feature `Y(t)` such as NIP density, MeaningGamma, TSP, MR, or ENC:

```
Y(t) = beta0 + beta_u^T U(t) + beta_art*Artifact(t) + eps(t)
```

where:

```
U(t) = [audio_env, luminance, visual_change, optical_flow, cut_train,
```

```
        flash_train, cue_on, cue_value, cue_phase_sin, cue_phase_cos, ALS_pulse]
```

Residual:

```
Y_resid(t) = Y(t) - Y_hat(t)
```

29.2 Blocked cross-validation

Split time into blocks, fit on training blocks, evaluate held-out blocks:

```
R2_blocked = 1 - sum((Y_test - Y_hat_test)^2) / sum((Y_test - mean(Y_train))^2)
```

If blocked-CV is poor or negative, the stimulus regressors do not generalize as an explanation.

29.3 Residualized CII

```
CII_resid(W_j) = mean_{t in W_j}(clip_plus(Y_resid(t)))
```

A robust Target effect should remain Target-dominant after residualization.

30. EET: Endogenous Echo Tracking

EET asks whether a stimulus-anchor state vector resembles later washout/Baseline2 physiology.

30.1 State vector

For anchor `j`:

```
StateVec_anchor,j = [MR, ENC, NIP, TSP, MeaningGamma, Theta, Alpha, API_A, HRV, Resp, ...]_Wj
```

For after-state window `q`:

```
StateVec_after,q = [same features]_Wq
```

30.2 Cosine similarity

```
EET_{j,q} = cosine(StateVec_anchor,j, StateVec_after,q)

          = dot(V_j, V_q) / (||V_j|| * ||V_q|| + eps)
```

30.3 Difference from control echo

```
DeltaEET_j = EET_{Target anchor j -> B2} - EET_{Control matched window j -> B2}
```

EET is state-vector resemblance. It is not proof of memory replay.

31. MRED-ITP: Information-Thermodynamic Proxy layer

MRED-ITP tests complexity perturbation/settlement and ocular release around MRED events.

31.1 ACG: Algorithmic Complexity Gate

Given symbolic sequence `S_t` derived from cleaned EEG window `W_t`:

```
C_LZ(t) = LZC(S_t) / E[LZC(shuffle(S_t))]
```

For anchor `j`:

```
C_pre,j = mean(C_LZ over [a_j - 15, a_j])

C_peak,j = mean(C_LZ over [a_j, a_j + 5])

C_post,j = mean(C_LZ over [a_j + 10, a_j + 30])



DeltaC_strike,j = C_peak,j - C_pre,j

DeltaC_settle,j = C_post,j - C_peak,j

DeltaC_baseline,j = C_post,j - C_pre,j
```

ACG flag:

```
ACG_j = 1[DeltaC_strike,j > theta_up]

      * 1[DeltaC_settle,j < -theta_down]

      * 1[Artifact_j < theta_artifact]
```

31.2 CSI: Complexity Settlement Index

```
CSI_j = z(DeltaC_strike,j) - z(DeltaC_post_minus_peak,j)
```

where `DeltaC_post_minus_peak` is positive when post remains above peak. A high CSI means perturbation followed by settlement.

31.3 OCU: Ocular-Cognitive Unloading

Blink detection proxy:

```
blink_i = 1[Fp1_p2p_i > theta_p2p

            and Fp1_lowfreq_i > theta_lf
```

```
            and duration_i in [100,500] ms]
```

Blink rates:

```
BR_pre,j = count(blinks in [a_j - 30, a_j - 5]) / 25

BR_hold,j = count(blinks in [a_j - 5, a_j + 5]) / 10

BR_release,j = count(blinks in [a_j + 5, a_j + 20]) / 15
```

Suppression and release:

```
BlinkSuppression_j = z(BR_baseline - BR_hold,j)

BlinkRelease_j = z(BR_release,j - BR_hold,j)
```

OCU index:

```
OCU_j = 0.45*z(BlinkSuppression_j)

      + 0.45*z(BlinkRelease_j)

      + 0.10*z(EventBoundary_j)

      - 0.25*z_positive(Artifact_j)
```

Strict OCU flag:

```
OCUFlag_j = 1[BlinkSuppression_j > theta_S]

          * 1[BlinkRelease_j > theta_R]

          * 1[Artifact_j < theta_artifact]
```

OCU is not proof of memory encoding.

32. LSO / SPM: Subtitle Override and Subtitle Phase Mapping

This is optional and not part of default PRAYCG2.0 unless selected.

32.1 Subtitle line representation

```
Sub_i = (t_on,i, t_off,i, text_i, bbox_i)
```

32.2 Lexical Extraction Cost

```
CharsPerSec_i = N_chars,i / (t_off,i - t_on,i)

WordsPerSec_i = N_words,i / (t_off,i - t_on,i)

LineLoad_i = N_lines,i
```

```
LEC_i = w_c*z(CharsPerSec_i)

      + w_w*z(WordsPerSec_i)

      + w_l*z(LineLoad_i)

      + w_s*z(SegmentationDifficulty_i)

      + w_g*z(GazeRevisits_i)
```

32.3 Subtitle semantic timing

If gaze is available:

```
t_read,i = first time gaze completes subtitle bbox traversal
```

If gaze is unavailable:

```
t_semantic,i = argmax_{t in [t_on,i - 2, t_off,i + 1]} TSP(t)
```

Phase shift relative to audio:

```
phi_i = t_read_or_semantic,i - t_audio_anchor,i
```

32.4 Subtitle MRED

```
MR_sub,j = 0.30*z(MeaningGamma_j)

         + 0.30*z(TSP_j)

         + 0.20*z(KHT_topo_j)

         + 0.20*z(LexicalRecognition_j)

         - 0.20*z(Artifact_j)



ENC_sub,j = 0.55*z(H_theta_j)

          + 0.15*z(API_A_j)

          + 0.15*z(Novelty_j)

          - 0.20*z(Familiarity_j)

          - 0.25*z(LEC_j)

          - 0.20*z(TaskGamma_j)

          - 0.15*z(Artifact_j)
```

Subtitle choke:

```
SubtitleChoke_j = 1[MR_sub,j > theta_MR]

                * 1[ENC_sub,j < theta_ENC]

                * 1[LEC_j > theta_LEC]
```

33. Topo-OSM Network State

Topo-OSM is the human-scale interpretation layer for network-state topology.

33.1 Human-scale latent state

```
Y_topo(t) = Phi[

    A_theta(t),

    A_gamma(t),

    A_alpha(t),

    TSP(t),

    API_A(t),

    Resp(t),

    Artifact(t),

    u(t)

]
```

This is not cellular `Y_cell` and not `K_OSM`.

33.2 State vector implementation

```
TopoState(t) = [ThetaTopology(t), GammaTopology(t), AlphaState(t),

                TSP(t), NIP(t), API_A(t), Resp(t), Artifact(t)]
```

33.3 Topological shift

For pre/post windows:

```
TopoShift_j = distance( mean(TopoState over W_post,j), mean(TopoState over W_pre,j) )
```

Possible distances:

```
Euclidean: ||V_post - V_pre||_2

Cosine:    1 - cosine(V_post, V_pre)

Mahalanobis: sqrt((V_post - V_pre)^T Sigma^-1 (V_post - V_pre))
```

33.4 Persistence

```
TopoPersistence_j = mean_{q in post windows}(cosine(V_anchor,j, V_q))
```

Topo-OSM does not prove memory. It provides a human-scale state-topology proxy.

34. Baseline 1 vs Baseline 2

34.1 Feature delta

For feature `F`:

```
DeltaF_B2_B1 = mean(F over B2) - mean(F over B1)
```

34.2 Reflective-state index

```
ReflectiveIndex = mean(
    z(DeltaTSP_B2_B1),
    z(DeltaNIP_B2_B1),
    z(DeltaAPI_A_B2_B1),
    z(-DeltaTaskGamma_B2_B1),
    z(-DeltaArtifact_B2_B1)
)
```

34.3 Regulation index

```
Baseline2Regulation = mean(
    z(-DeltaHR_B2_B1),
    z(DeltaRMSSD_B2_B1),
    z(DeltaSDNN_B2_B1),
    z(DeltaPNN50_B2_B1),
    z(-DeltaRespInstability_B2_B1)
)
```

These feed MRED-Resolution and NUPI.

35. SelfReport20 parser

Self-report is an independent evidence stream, not proof.

35.1 Branch core report vector

```
Report_branch = [Meaning, Absorption, EmotionalAfterglow, StoryActiveWashout,

                 TaskExtractionLoad, ConfoundBurden]
```

35.2 Final report vector

```
Report_final = [OverrideReducedReception, StoryBrokeThroughOverride,

                TargetEchoedDuringBaseline2, Familiarity, NewMeaningToday,

                CurrentLifeResonance]
```

35.3 Model relation

A generic mixed model:

```
Report_{i,k} = alpha

             + beta_MR * MR_{i,k}

             + beta_ENC * ENC_{i,k}

             + beta_NIP * NIP_{i,k}

             + beta_API * API_A_{i,k}

             - beta_X * TaskGamma_{i,k}

             - beta_A * Artifact_{i,k}

             + u_subject

             + eps_{i,k}
```

A positive correspondence is not proof of internal state; contradiction is a warning.

36. Visualizer formulas

The visualizer does not create scientific endpoints. It audits synchronization and module outputs.

36.1 Robust axis scaling

For visible panel window `W_vis`:

```
y_min = percentile(y over W_vis, 5)

y_max = percentile(y over W_vis, 95)
```

or z-score mode:

```
y_plot = clip(rz(y), -z_clip, z_clip)
```

```
```

36.2 Missing-signal gate

```
UsableFeature = 1[non_null_count >= N_min]

              * 1[std(y) > eps]

              * 1[unique_count > 1]
```

If `UsableFeature = 0`, the panel should display “signal unavailable” rather than a fake flat line.

36.3 Event card display

For event time `e_k`:

```
ShowEventCard_k(t) = 1[ |t - e_k| < card_window ]
```

37. Offline interpretive report generator

The offline interpreter is deterministic and rule-based.

37.1 Table detection

```
ModulePresent_m = 1[expected_table_m exists]
```

37.2 Rule evaluation

Example:

```
if A_MRED_pass_count > 0 and claim_level strict:

    statement = "A-MRED positive under current strict gate."

elif A_MRED_pass_count > 0 and timing_caution:

    statement = "A-MRED pilot-positive with timing caution."

else:

    statement = "No strict A-MRED pass detected."
```

37.3 Boundary

```
The offline interpreter explains tables.

It does not create new evidence.
```

```
It does not replace expert review.
```

38. Opto-PING / Rung 0 synthetic model

Opto-PING is synthetic identifiability and model-comparison support. It does not prove human biology.

38.1 Orthodox PING skeleton

```
E(t) <-> I(t)
```

where:

```
E(t) = excitatory population activity

I(t) = inhibitory population activity
```

38.2 Hidden reciprocal model

```
E_{t+1} = a_E*E_t + eta_E*Y_t + b_E*u_t + eps_E,t

Y_{t+1} = a_Y*Y_t + zeta_E*E_t + b_Y*u_t + eps_Y,t
```

38.3 Loop coefficient

```
K_loop = eta_E * zeta_E

K = sqrt(abs(eta_E * zeta_E))
```

K is more stable than separately interpreting eta and zeta.

38.4 Joint negative log likelihood

For conditions `c = 1..C` and Kalman innovations `v_k^(c)` with covariance `S_k^(c)`:

```
NLL(theta) = sum_c sum_k [ log|S_k^(c)| + v_k^(c)^T * inv(S_k^(c)) * v_k^(c) ]
```

where:

```
theta = [eta_E, zeta_E, process noise, observation noise, ...]
```

38.5 Model-selection gate

```
Pass_Rung0 = 1[K_error <= threshold]

           * 1[FullModel_CV_WinRate >= threshold]

           * 1[NullAlternatives do not match full model]
```

Boundary:

```
Synthetic identifiability only.

No biological or human EEG mechanism claim.
```

39. Module interaction map

A single event can pass through the suite as:

```
1. MediaPrep creates stimulus + cue + anchor scaffold.

2. Runner records which files were used and writes Stasis markers.

3. Feature extractor computes EEG/autonomic/stimulus features.

4. ArtifactScore and confound registry define veto/caution layers.

5. TSP/GammaScalpel detect candidate recognition timing.

6. Theta/Topo/after-state features estimate integration.

7. MRED separates recognition from integration.

8. A-MRED applies the strict primary endpoint gate.

9. NIP/CII summarizes continuous immersion density.

10. TTI compares reception vs extraction.

11. NUPI classifies update polarity.

12. CET-R tests whether stimulus rhythm explains the effect.

13. EET/MRED-ITP/OCU/ACG provide exploratory convergence.

14. Visualizer and offline interpreter explain the result without adding evidence.
```

40. Example event derivation

For a predeclared Target anchor `j`:

```
Step 1: compute MeaningGamma_j and TSP_j.

Step 2: compute MR_T,j.

Step 3: compute H_theta_10_30,j, TopoShift_j, API_A_post,j.

Step 4: compute ENC_T,j.

Step 5: repeat matched windows for Control and Override.

Step 6: compute ArtifactPass, ConfoundPass, TimingPass, CETRPass.

Step 7: compute A_MRED_j.

Step 8: compute CII_T/C/O and IAQ.
```

```
Step 9: compute PeakEvidence and ResolutionEvidence.

Step 10: write visual overlay and offline interpretation.
```

Mathematically:

```
A_MRED_j = Gate(MR_T,j, ENC_T,j, MR_C,j, ENC_C,j, MR_O,j, ENC_O,j, QC_j)


CII_margin_j = CII_T,j - max(CII_C,j, CII_O,j)


PeakEvidence_j = f(MR_T,j, ENC_T,j, CII_margin_j, KHT_topo_j, QC_j)


ResolutionEvidence_j = f(B2_echo_j, EET_j, RDI_j, self_report_echo_j, API_recovery_j)
```

41. Formula safety notes

41.1 The suite contains operational indices

These formulas are engineered proxies. They are not direct measurements of hidden essence.

```
MeaningGamma != meaning

TSP != true semantic time

KHT-topo != K_OSM

EET != replay proof

OCU != memory dump proof

NUPI != heat or ATP

TTI != moral theft

A-MRED != consciousness proof
```

41.2 Positive result language

Use:

```
Target-specific recognition-plus-integration candidate.

A-MRED pass under current gates.

MRED-Peak candidate.

MRED-Resolution candidate.

Resolvable recovery profile.
```

```
Accommodative-load profile.
```

Avoid:

```
proved consciousness

proved memory formation

proved OSM biology

proved the soul

proved thermodynamic meaning
```

41.3 Negative result language

Use:

```
No strict pass under current operational criteria.

Recognition may have occurred without detectable integration.

Timing or artifact caveats prevent strict interpretation.
```

Avoid:

```
nothing happened

the scene was meaningless

the subject felt nothing
```

42. Minimal confirmatory endpoint recommendation

For a future group or preregistered study, the primary endpoint should be one of:

```
A-MRED pass proportion across locked anchors

DeltaMRED continuous score across locked anchors

MRED-Peak vs MRED-Resolution classification across locked anchors
```

The confirmatory statistical model could be:

```
A_MRED_pass ~ Condition + (1|Subject) + (1|Stimulus) + (1|Anchor)
```

or continuous:

```
DeltaMRED ~ Condition + Familiarity + Artifact + CETR + (1|Subject) + (1|Stimulus) + (1|Anchor)
```

Prediction:

```
Target > Control
```

```
Target > Override
```

The rest of the suite should be treated as secondary or exploratory unless preregistered as primary.

43. Appendix A: recommended output files by module

```
Core feature extraction:
  *_time_resolved_feature_frame.csv
  *_phase_feature_summary.csv


A-MRED:
  amred_anchor_endpoint_table.csv
  amred_primary_endpoint_summary.csv
  amred_visual_overlay.csv


MRED-Peak/Resolution:
  mred_peak_resolution_anchor_table.csv
  mred_peak_resolution_run_summary.csv
  top_mred_peak_candidates.csv
  top_mred_resolution_candidates.csv
  mred_peak_resolution_visual_overlay.csv


MRED / KHT:
  mred_event_table.csv
  candidate_local_kht_topo_mred_event_table.csv
  candidate_local_kht_visual_overlay.csv


NIP / CET / EET:
  nip_component_timeseries.csv
  bit_event_table.csv
```

```
  cii_anchor_integrals.csv
  iaq_target_override_table.csv
  cet_residualization_model_summary.csv
  eet_endogenous_echo_tracking.csv

TTI / NUPI:
  tti_global_summary.csv
  tti_anchor_deltas.csv
  nupi_run_summary.csv
  nupi_anchor_polarity_table.csv

OCM/RSM/CVB/Squint:
  ocm025_rsm_cvb_squint_cue_epoch_table.csv
  rsm_correlation_summary.csv
  confound_registry.csv

MRED-ITP:
  lz_complexity_timeseries.csv
  acg_event_table.csv
  blink_event_table.csv
  ocu_event_table.csv
  mred_itp_anchor_summary.csv

Visualizer:
  *_features_used.csv
  *_events_used.csv
  *_feature_diagnostics.csv
  *_event_time_diagnostics.csv
  *_render_report.json

Offline interpreter:
  reports/offline_interpretation/offline_interpretive_report.md
```

```
  reports/offline_interpretation/offline_interpretive_report.txt

  reports/offline_interpretation/offline_interpretive_report_summary.json
```

44. Appendix B: file provenance rule

```
MediaPrep:

  stimulus videos, cue schedules, draft/locked anchor scaffolds, stimulus fingerprints.



Protocol runner:

  event markers, media file hashes, run configuration, self-reports, confound reports.



Master Comprehensive Suite:

  feature tables, module outputs, event tables, endpoint summaries.



Visualizer:

  synchronized MP4 and sidecar tables.



Offline interpreter:

  deterministic text interpretation of existing module outputs.
```

Clean rule:

```
If a file describes what was shown, it probably comes from MediaPrep or the runner.

If a file describes physiology over time, it comes from the Master Comprehensive Suite.

If a file explains what the outputs mean, it comes from the offline interpreter or report generator.
```

45. Appendix C: compact formula catalog

```
rz(x) = (x - median_ref) / (1.4826*MAD_ref + eps)



PLV = |mean(exp(i*(phase_a - phase_b)))|
```

```
ArtifactScore = mean(rz(p2p), rz(HF_45_55), rz(EMG), rz(FpSentinel), rz(line_noise), step_flag)


API_A = .30*z(-HR) + .25*z(RMSSD) + .20*z(SDNN) + .15*z(pNN50) + .10*z(-HR_slope) -
.10*z(RespInstability)


TSP = w_G*MeaningGamma + w_PT*PosteriorTemporal + w_N*NAST + w_K*KHT - w_X*TaskGamma - w_A*Artifact -
w_V*VisualDrive


KHT_topo = sqrt(abs(eta_HT*zeta_HT))


MR = w_TSP*TSP + w_G*MeaningGamma + w_K*KHT + w_N*NAST - penalties


ENC = w_H*ThetaHandoff + w_T*TopoShift + w_E*EET + w_API*API_A + w_B2*B2Echo - penalties


A_MRED = 1[MR_T>th] * 1[ENC_T>th] * 1[T>C] * 1[T>O] * 1[QC=PASS]


NIP_density = clip_plus(A_sem)^alpha * clip_plus(R_int_lag)^beta * Penalty


CII = mean(NIP_density over anchor window)


IAQ = 1 - (CII_Override + eps)/(CII_Target + eps)


TTI = w1*z(MR_T-MR_O)+w2*z(ENC_T-ENC_O)+w3*z(API_T-API_O)+w4*z(X_O-X_T)-penalties


NUPI = RDI - ALI


CET residual: Y_resid = Y - (beta0 + beta_u^T U + beta_art*Artifact)


EET = cosine(StateVec_anchor, StateVec_after)


LZC_norm = LZC(S)/E[LZC(shuffle(S))]
```

```
OCU = .45*z(BlinkSuppression)+.45*z(BlinkRelease)+.10*z(EventBoundary)-.25*z(Artifact)
```

```
LEC = w_c*z(chars/sec)+w_w*z(words/sec)+w_l*z(line_count)+w_s*z(segmentation)+w_g*z(gaze_revisits)
```

```
OptoPING K = sqrt(abs(eta_E*zeta_E))
```

46. Final statement

The Master Comprehensive Analysis Suite is best understood as a layered falsification machine. Its most useful result is not that every module produces a positive number. Its most useful result is that the same event can be tested against timing, artifact, sensory entrainment, task extraction, self-report, autonomics, and state persistence before the language of meaning is allowed into the report.

The formulas in this document give each layer a clear mathematical address. They should be treated as auditable operational definitions, not as metaphysical declarations.